data to make such decisions (just like we need a lot of experience). If this text came from a speech recognition system, inflection could be an additional context.

Natural language processing then needs specialised tools that are built on data, the more the merrier. In this article we will look at a rather popular open source toolkit for natural language processing called NLTK (Natural Language ToolKit). Since we don't want Google to go out of business with our cool new language processing software, we will only focus on the very basics. We'll focus on taking natural language as input, and discovering things about that text programmatically.

## Getting, installing and setting up NLTK

NLTK is a Python package that includes a large number of features that have to do with managing, cleaning, importing and processing text. Since it is a Python package, you will need to have Python installed in order to use it.

To keep things simple we recommend that you simply download and install Anaconda, a Python distribution by Continuum Analytics. This Python distribution is available for Windows, macOS and Linux and includes not only NLTK but a number of other popular Python libraries, frameworks and packages that could be of use.

You will need the Python 3.5 version of the Anaconda distribution, 32-bit or 64-bit doesn't matter as much. https://www.continuum.io/

It's a large download since it includes a lot of libraries. It's also possible to download a minimal bare Python version, and install just what you need. The minimal version is called Miniconda and is about a tenth the size of the complete package. If you pick this option though you will need to install NLTK manually using `conda install nltk`.

If you already have a recent version of Python 3 installed, you can still try Anaconda / Miniconda, or just install NLTK using `pip install –user nltk` in your terminal. The `–user` ensures that the package is installed just for the current user so you don't need to use sudo or obtain admin privileges.

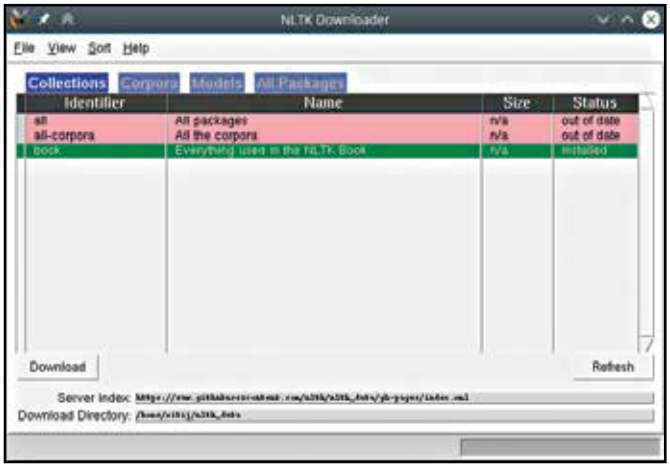
There is one more step. While NLTK includes a number of great tools for processing text, it doesn't include the data those algorithms need to run. There is gigabytes of data available and most people don't need all of it. This data is what uses to actually understand language. For instance it doesn't include a dictionary or other pieces of data necessary for understanding natural language.

NLTK includes a handy tool for downloading this data which you can launch from a Python shell. We will use ipython an enhanced Python shell, since it features code completion and other enhanced features not found in the standard shell available via python. If you installed Anaconda, you can also try jupyter qtconsole for a Python shell that has a GUI, and jupyter notebook which lets you interact with the Python shell in a browser.

If you are using your own Python install, or installed Miniconda, you can install ipython and other nice tools using conda install jupyter for Miniconda, and pip install –user Jupyter otherwise. In any case you can launch one of these shells by typing them in the terminal.

Now you can type the following in the Python shell (whichever one you use):

```
import nltk
nltk.download()
```




NLTK includes a GUI and a console tool for downloading essential data needed for its operation.

A dialog should pop up that lets you pick the data you want to download. Select "book" since it has a good collection of the data you will need, and it small enough at under 100MB. You can browse other tabs to see what all is available. If at any point you get errors about missing data, you can use this to get it.

## Tokenizing text

Tokenizing sounds fancy but it's simply the act of splitting text into smaller elements (tokens). We might split some text into paras, or sentences, or words. It's easy to understand, but harder to actually implement. It might seem simple to split by sentence till you realize that you'll need to look out for cases like "Dr." or "Mr.". Likewise, splitting at words is more complex than just splitting at each space character, especially considering cases like "cannot" and "wouldn't" which should split, are both contractions of two words.

NLTK includes a collection of tokenisers that can handle all kinds of special cases, not just words, and sentences. These is also a TweetTokeniser that can handle emoji in Tweets.

The "book" collection we downloaded in the previous section includes the entire text of a few public domain books. Let's open one of them and tokenise it by words.

```
import nltk
persuasion_file = nltk.corpus.gutenberg.open('austen-persuasion.
txt')
persuasion_text = persuasion_file.read()
persuasion_words = nltk.word_tokenize(persuasion_text)
```

In our examples we will use "Persuasion" by Jane Austen. Here we have first opened the included copy of Persuasion which is part of the Gutenberg corpus. We then read in the entire text of that file, and use the included word tokeniser to break the text into words.

Try out `nltk.word_tokenize('...')` on some of your own sentences to see how it handles different cases. You can also see a list of other tokenisers typing `dir(nltk.tokenize)`. If you want more information about one of them you can use the help function. For example: `help(nltk.tokenize.TreebankWordTokenizer)`

## Exploring our Text

To handle large bundles of text, like this entire 250-page book, NLTK had a class called Text. We'll wrap our persuasion_words data in a Text object to explore it with greater ease.

```
persuasion = nltk.Text(persuasion_words)
```